

EASA COLLEGE
OF ENGINEERING & TECHNOLOGY (ECET)
APPROVED BY AICTE, NEW DELHI | AFFILIATED TO ANNA UNIVERSITY, CHENNAI
AN AUTONOMOUS INSTITUTION



ROLE OF INFORMATION EXTRACTION IN BUILDING KNOWLEDGE GRAPHS

A CASE STUDY ON GOOGLE KNOWLEDGE GRAPH

NER • RELATION EXTRACTION • TRIPLES • KNOWLEDGE GRAPH

A TERM PROJECT REPORT

Submitted by

RAKAVI BALA

Register No: E720524AM020

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

(ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

EASA COLLEGE OF ENGINEERING & TECHNOLOGY

ANNA UNIVERSITY : CHENNAI 600 025

OCTOBER 2026

ANNA UNIVERSITY :: CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this term project report "ROLE OF INFORMATION EXTRACTION IN BUILDING KNOWLEDGE GRAPHS: A CASE STUDY ON GOOGLE KNOWLEDGE GRAPH" is the bonafide work of "RAKAVI BALA" who carried out the term project work under my supervision.

SIGNATURE

DR. M G DINESH
HEAD OF THE DEPARTMENT
Department of Artificial Intelligence and Data Science
EASA COLLEGE OF ENGG & TECH,
COIMBATORE - 641 105

SIGNATURE

MR. C G KIRUBHANANDHAN
ASSISTANT PROFESSOR
Department of Artificial Intelligence and Data Science
EASA COLLEGE OF ENGG & TECH,
COIMBATORE - 641 105

ACKNOWLEDGEMENT

I express my sincere gratitude to the management and the Principal of EASA College of Engineering & Technology, Coimbatore, for providing the facilities required to complete this term project.

I am grateful to Dr. M G Dinesh, Head of the Department of Artificial Intelligence and Data Science, for his encouragement and support throughout the course.

I record my heartfelt thanks to my faculty guide, Mr. C G Kirubhanandhan, Assistant Professor, for his guidance in the subject Text Analytics and for the opportunity to study the role of information extraction in knowledge graphs.

I also thank my parents, friends and classmates for their continuous help and motivation during this work.

RAKAVI BALA

TABLE OF CONTENTS

Abstract..... 5
1. Introduction..... 5
2. Information Extraction and Knowledge Graphs..... 6
3. Methodology..... 11
4. Results..... 13
5. Case Study: Google Knowledge Graph..... 18
6. Limitations and Discussion..... 20
7. Conclusion..... 21
8. Project Links..... 21
9. References..... 21
Appendix A: Complete Source Code..... 23
Appendix B: Repository Structure and How to Run..... 24
Appendix C: Glossary..... 25

LIST OF FIGURES

Figure 1: Information extraction pipeline used to build a knowledge graph..... 7
Figure 2: Entity mentions found by spaCy, grouped by label..... 14
Figure 3: Knowledge graph built from the seven curated triples (Matplotlib output)..... 15
Figure 4: Strict and lenient scores of the rule-based extractor..... 16
Figure 5: Sources feeding the Google Knowledge Graph (simplified)..... 19

Abstract

Knowledge graphs (KGs) store facts as entities connected by relations and power modern search, question answering and recommendation systems. Most real-world knowledge is stored in unstructured text, so Information Extraction (IE) is the key step that converts text into structured knowledge. This project builds a small end-to-end pipeline in Python using spaCy, NetworkX and PyVis that performs Named Entity Recognition, relation extraction, triple generation and graph visualisation. The pipeline is then compared with the Google Knowledge Graph, a large-scale real-world KG, to study the role, strengths and limitations of information extraction in building knowledge graphs.

1. Introduction

A knowledge graph represents information as a network in which nodes are entities (people, organisations, places) and edges are relationships between them. Each fact is stored as a triple of the form (subject, relation, object), for example (Larry Page, founded, Google). Google introduced its Knowledge Graph in 2012 with the idea of understanding "things, not strings", which means that a search engine should understand real-world entities instead of only matching keywords.

The text on the web is unstructured, so it cannot be placed directly into a graph. Information Extraction is the group of Text Analytics techniques that identifies entities and relations in raw text and converts them into structured triples. Without IE, large knowledge graphs could not be built or kept up to date.

1.1 Objectives

- To explain the role of information extraction in building knowledge graphs.
- To implement a working pipeline: text, NER, relation extraction, triples and graph.
- To study the Google Knowledge Graph as a real-world case study.
- To compare the small pipeline with Google KG and discuss limitations.

1.2 Motivation

Search engines, voice assistants and recommendation systems are expected to answer questions about real-world things, not only to return a list of web pages. When a person asks "Who founded Google?", the system must know that Google is a company, that Larry Page and Sergey Brin are people and that the relation between them is "founded". Such knowledge is kept in a knowledge graph. Building one manually is impossible at web scale because the facts are spread over billions of sentences, tables and web pages. Information extraction makes it possible to collect this knowledge automatically.

This project is motivated by the need to understand how the individual steps of information extraction (entity recognition, relation extraction and triple generation) contribute to the final knowledge graph, and how a small, transparent pipeline differs from an industrial system such as the Google Knowledge Graph.

1.3 Problem Statement

Given unstructured English text, automatically identify the entities and the relations between them, represent each fact as a (subject, relation, object) triple, and assemble the triples into a knowledge graph. Then analyse how the quality of the extraction affects the graph, using the Google Knowledge Graph as a real-world reference.

1.4 Scope of the Project

- Language: English text only.
- Input: a short, hand-written text about Google (five sentences).
- Methods: pre-trained spaCy model for NER and rule-based dependency parsing for relation extraction.
- Output: triples, a static graph (Matplotlib) and an interactive graph (PyVis).
- Case study: Google Knowledge Graph, studied from published material; the project does not use any private Google data.

1.5 Organisation of the Report

Chapter 1 introduces the topic. Chapter 2 explains information extraction and knowledge graphs in detail. Chapter 3 describes the methodology and the implementation. Chapter 4 presents the results, evaluation and error analysis. Chapter 5 is the case study on the Google Knowledge Graph. Chapter 6 discusses limitations, ethical points and future work. Chapter 7 concludes the report. Chapters 8 and 9 give the project links and references, and the appendices contain the complete source code and the instructions to run it.

2. Information Extraction and Knowledge Graphs

2.1 Main IE tasks

- Named Entity Recognition (NER): finding and labelling entities such as PERSON, ORG, GPE and DATE.
- Entity Linking / Disambiguation: mapping a mention such as "Google" to the correct unique entity in the KG.
- Relation Extraction: identifying how two entities are related, for example founded, acquired, CEO of.
- Triple generation: writing each fact as (subject, relation, object).

- Knowledge fusion: merging facts from many sources and removing duplicates and conflicts.

2.2 How IE feeds a knowledge graph

The flow is: raw text, then NER, then entity linking, then relation extraction, then triples, then the knowledge graph. The quality of the graph depends directly on the accuracy of each earlier stage. An error in NER or in relation extraction becomes a wrong or missing edge in the graph.

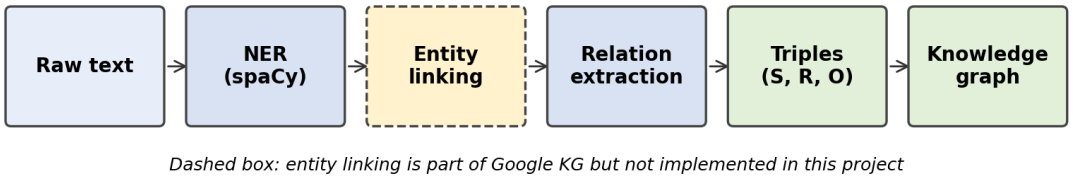


Figure 1: Information extraction pipeline used to build a knowledge graph

2.3 Knowledge Graph Data Models

A knowledge graph can be stored in more than one way. The simplest view is a set of triples. In the RDF (Resource Description Framework) model, every fact is a triple of subject, predicate and object, and every resource is identified by a unique URI, so that "Google" in two different data sets can be recognised as the same thing. Graph databases use the property graph model, in which nodes and edges can carry extra properties such as dates or confidence scores. In this project the directed graph class of NetworkX is used, which behaves like a property graph: each edge stores a label attribute holding the relation name.

Table 1: Examples of facts stored as triples

Subject	Predicate	Object
Google	foundedBy	Larry Page
Google	headquarters	Mountain View
Mountain View	locatedIn	California
Google	acquired	YouTube (year 2006)

2.4 Ontologies and Schema

A knowledge graph needs a schema, also called an ontology, which lists the types of entities (Person, Organization, Place) and the relations that may exist between them (for example, a Person can be the founder of an Organization). The schema keeps the graph consistent and allows reasoning: if X is the CEO of an Organization, then X is a Person. The collaborative schema.org vocabulary, started in 2011 by Google, Bing, Yahoo and Yandex, is widely used by

web sites to describe their content in a machine-readable way, and search engines use this markup as one of the sources of structured data.

2.5 Well-Known Knowledge Graphs

Table 2: Some well-known knowledge graphs

Name	Origin / year	Main characteristics
Freebase	Metaweb, 2007 (bought by Google in 2010)	Community edited; later closed and its data moved to Wikidata
DBpedia	Univ. of Leipzig and Berlin, 2007	Facts extracted automatically from Wikipedia infoboxes
YAGO	Max Planck Institute, 2007	Combines Wikipedia with WordNet taxonomy
NELL	Carnegie Mellon Univ., 2010	Never-ending learner that reads the web continuously
Wikidata	Wikimedia Foundation, 2012	Free, collaboratively edited, multilingual KG
Google Knowledge Graph	Google, 2012	Powers knowledge panels and answers in Google Search

2.6 Applications of Knowledge Graphs

- Search: knowledge panels, direct answers and related searches.
- Question answering and virtual assistants: converting a question into a graph query.
- Recommendation: linking users, items and attributes to explain suggestions.
- Enterprise data integration: connecting data from many databases through shared entities.
- Biomedicine and science: linking genes, drugs, diseases and publications.
- Fraud and risk analysis: finding hidden connections between people and companies.

2.7 Techniques of Information Extraction

2.7.1 Named Entity Recognition

NER marks spans of text that refer to real-world objects and gives each span a type. Early systems used hand-written rules and dictionaries (gazetteers) of names. Statistical systems such as Hidden Markov Models and Conditional Random Fields learn from labelled text using features like word shape, capitalisation and neighbouring words. Neural systems (BiLSTM-CRF) and transformer models (BERT and its successors) read the whole sentence and give the best accuracy. The small spaCy model used here is a convolutional neural network trained on the OntoNotes corpus, with labels such as PERSON, ORG, GPE (geo-political entity), DATE and PRODUCT.

2.7.2 Entity Linking and Disambiguation

After a mention such as "Google" has been found, entity linking decides which entry of the knowledge base it refers to. The usual steps are candidate generation (find all entries that could match), ranking (use the surrounding context and the popularity of each candidate) and a decision to create a new entry when no candidate fits. Linking solves two problems: the same entity may appear with different names ("Alphabet's Google", "Google LLC"), and the same name may refer to different entities ("Mountain View" the city and other places with that name). This step is part of the Google Knowledge Graph pipeline but is not implemented in this project.

2.7.3 Relation Extraction

Relation extraction decides whether two entities in a sentence are connected and by which relation. The main approaches are given below.

Table 3: Approaches to relation extraction

Approach	**Idea**	**Strength / weakness**
Pattern based	Hand-written lexical or syntactic patterns, e.g. X founded Y	Precise but covers few sentences
Supervised learning	Classifier trained on labelled sentences	Accurate; needs costly labelled data
Distant supervision	Align a KG with text to create training data automatically	Cheap; training labels are noisy
Open IE	Extract any (arg1, relation phrase, arg2) without a fixed schema	Wide coverage; relations are not normalised
Transformer based	Fine-tune a language model on entity pairs	Best accuracy; needs more computing power

The method used in this project is a syntactic pattern: the dependency parse of the sentence is used to find a verb with a subject on its left and an object on its right. This is the simplest form of pattern-based extraction and is easy to explain, but it covers only simple active sentences.

2.7.4 Coreference Resolution

Texts often refer to an entity with a pronoun or a short name ("The company", "it"). Coreference resolution links these mentions to the full entity so that facts are not lost. The input text of this project avoids pronouns, so coreference is not needed here, but it is a major step for real documents.

2.7.5 Knowledge Fusion

The same fact can be extracted from many sources, and different sources can disagree. Knowledge fusion merges duplicate facts, estimates the reliability of each source and extractor,

and keeps a confidence score for every fact. Only facts with sufficient confidence are added to the graph.

2.7.6 Evaluation Measures

The quality of extraction is measured against a set of correct facts. Precision is the fraction of extracted facts that are correct, recall is the fraction of correct facts that were found, and the F1 score is their harmonic mean:

```
Precision = correct extracted / all extracted
Recall    = correct extracted / all correct (gold)
F1        = 2 x Precision x Recall / (Precision + Recall)
```

3. Methodology

3.1 Tools and environment

Tool	Purpose
Python 3 / Google Colab	Development environment
spaCy (en_core_web_sm)	Tokenisation, NER, dependency parsing
NetworkX	Building the directed graph
Matplotlib	Static graph visualisation
PyVis	Interactive graph visualisation

3.2 Input text

A short text about Google was used as input:

```
Google was founded by Larry Page and Sergey Brin in 1998.
Google is headquartered in Mountain View, California.
Sundar Pichai is the CEO of Google.
Google acquired YouTube in 2006.
Google launched the Knowledge Graph in 2012.
```

3.3 Step 1: Named Entity Recognition

The spaCy model en_core_web_sm was loaded and applied to the text. The model identified the entities listed in Section 4.1.

3.4 Step 2: Relation extraction

Two approaches were used. First, a curated list of triples was prepared from the sentences. Second, an automatic rule-based extractor was written using spaCy dependency parsing: for every verb, the subject (nsubj / nsubjpass) on its left and the object (dobj / attr) on its right were taken, and the verb lemma was used as the relation.

3.5 Step 3: Graph construction and visualisation

A directed graph was built with NetworkX, where each triple adds an edge from subject to object with the relation as the edge label. The graph was drawn with Matplotlib and also as an interactive, draggable graph with PyVis.

3.6 Internal Working of the spaCy Pipeline

When nlp(text) is called, spaCy passes the text through a sequence of components. The tokenizer splits the text into tokens; the tagger assigns part-of-speech tags such as VERB and NOUN; the dependency parser links every token to its head with a label such as nsubj (nominal subject) or dobj (direct object); the lemmatizer reduces words to their base form ("acquired"

becomes "acquire"); and the named entity recogniser marks entity spans and their labels. Because relation extraction in this project depends on the part-of-speech tags and dependency labels, its accuracy is directly limited by the accuracy of the small English model (en_core_web_sm).

3.7 Rule-Based Relation Extraction Algorithm

The automatic extractor follows the steps given below.

- Step 1: Split the document into sentences using doc.sents.
- Step 2: For every token in the sentence, check whether it is a verb.
- Step 3: Collect the words on the left of the verb whose dependency label is nsubj or nsubjpass (subject).
- Step 4: Collect the words on the right of the verb whose dependency label is dobj or attr (object).
- Step 5: If both a subject and an object exist, store the triple (subject, verb lemma, object).

Only the head word of each phrase is taken, which explains why the multi-word name "Knowledge Graph" is reduced to a single word. The core of the implementation is given below.

```
auto_triples = []
for sent in doc.sents:
    for token in sent:
        if token.pos_ == "VERB":
            subj = [w.text for w in token.lefts if w.dep_ in ("nsubj",
"nsubjpass")]
            obj  = [w.text for w in token.rights if w.dep_ in ("dobj", "attr")]
            if subj and obj:
                auto_triples.append((subj[0], token.lemma_, obj[0]))
```

3.8 Graph Construction

Each curated triple is added to a NetworkX DiGraph as a directed edge from the subject node to the object node, and the relation is stored in the edge attribute label. The spring layout algorithm places connected nodes close to each other, and the labels are drawn on the edges. For the interactive version, PyVis converts the same triples into an HTML page using the vis.js library, in which nodes can be dragged with the mouse.

```
G = nx.DiGraph()
for s, r, o in triples:
    G.add_edge(s, o, label=r)
pos = nx.spring_layout(G, seed=42, k=1.5)
nx.draw(G, pos, with_labels=True, node_color="skyblue", node_size=3500)
nx.draw_networkx_edge_labels(G, pos, edge_labels=nx.get_edge_attributes(G,
"label"))
```

3.9 Evaluation Method

The seven curated triples are treated as the gold standard because they were written by reading the five sentences. The automatically extracted triples are compared with them in two ways. Under the strict measure a triple is correct only if subject, relation and object match exactly. Under the lenient measure a triple is also accepted when the relation and the head word of the object are correct, even if the object name is incomplete.

3.10 Walkthrough on One Sentence

To make the working of the extractor clear, the sentence "Google acquired YouTube in 2006." is followed through the pipeline. spaCy tokenises it, tags each token and finds the dependency label of each token.

Table 4: Linguistic analysis of the sentence Google acquired YouTube in 2006

Token	POS tag	Dependency	Role in extraction
Google	PROPN	nsubj	Subject of the triple
acquired	VERB	ROOT	Relation (lemma: acquire)
YouTube	PROPN	dobj	Object of the triple
in	ADP	prep	Not used
2006	NUM	pobj	Date (entity label DATE)

The verb acquired has Google on its left with the label nsubj and YouTube on its right with the label dobj, so the rule produces the triple (Google, acquire, YouTube). The same sentence is handled correctly by the entity recogniser, which labels Google and YouTube as ORG and 2006 as DATE.

4. Results

4.1 Named entities found

Entity	Label	Meaning
Google, YouTube, the Knowledge Graph	ORG	Organisation
Larry Page, Sergey Brin, Sundar Pichai	PERSON	Person
Mountain View, California	GPE	Geo-political entity
1998, 2006, 2012	DATE	Date

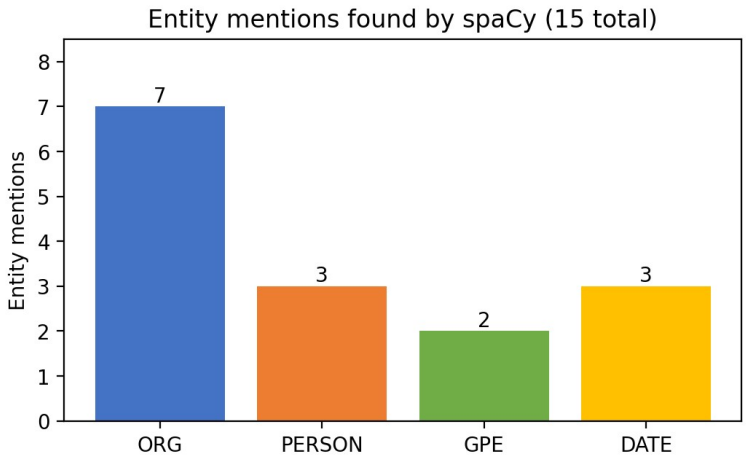


Figure 2: Entity mentions found by spaCy, grouped by label

In total 15 entity mentions were found in the five sentences. The company name Google appears five times (once in each sentence), which is why ORG is the largest group. All persons, places and dates in the text were identified correctly by the model.

4.2 Triples (curated)

Subject	Relation	Object
Larry Page	founded	Google
Sergey Brin	founded	Google
Google	headquartered in	Mountain View
Mountain View	located in	California
Sundar Pichai	CEO of	Google
Google	acquired	YouTube
Google	launched	Knowledge Graph

4.3 Triples (automatic extraction)

Subject	Relation	Object
Google	acquire	YouTube
Google	launch	Graph

The automatic extractor found only 2 of the 7 facts, and one of them is incomplete because "Knowledge Graph" was reduced to "Graph".

4.4 Knowledge graph output

The final graph contains 8 nodes (Google, Larry Page, Sergey Brin, Sundar Pichai, YouTube, Knowledge Graph, Mountain View and California) connected by labelled directed edges. Google is the central node, linked to its founders, CEO, headquarters, an acquisition and a

product. The static graph (Matplotlib) and the interactive graph (PyVis) screenshots are attached in the output screenshots PDF and in the GitHub repository.

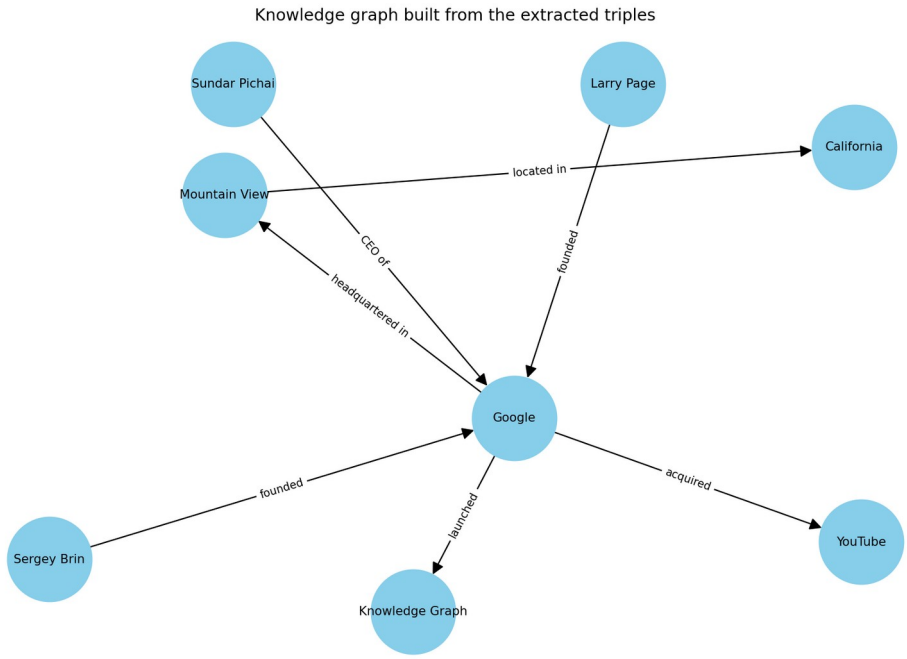


Figure 3: Knowledge graph built from the seven curated triples (Matplotlib output)

Figure 3 shows the graph. Google is the hub with five connections: two incoming "founded" edges from the founders, one incoming "CEO of" edge, one outgoing "headquartered in" edge, one "acquired" edge to YouTube and one "launched" edge to the Knowledge Graph. Mountain View is connected to California by the "located in" edge, so the graph also answers a two-step question such as "In which state is Google headquartered?" by following Google, Mountain View and California.

4.5 Quantitative Evaluation of Automatic Extraction

The automatic extractor returned two triples, ('Google', 'acquire', 'YouTube') and ('Google', 'launch', 'Graph'), while the gold standard has seven triples.

Table 5: Precision, recall and F1 of the automatic extractor

Measure	Precision	Recall	F1
Strict matching	0.50 (1/2)	0.14 (1/7)	0.22
Lenient matching	1.00 (2/2)	0.29 (2/7)	0.44

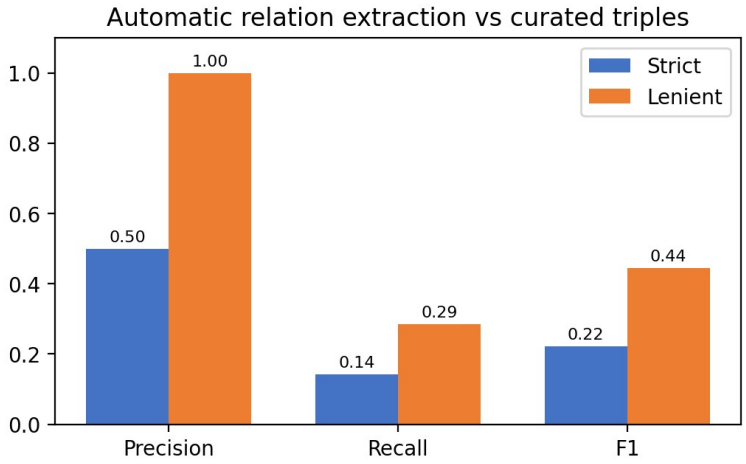


Figure 4: Strict and lenient scores of the rule-based extractor

The scores show that the extractor is precise when it produces a result but misses most of the facts. The low recall is the main weakness of simple rule-based extraction.

4.6 Error Analysis

Table 6: Sentence-wise analysis of the automatic extraction

Sentence	Extracted?	Reason
Google was founded by Larry Page and Sergey Brin in 1998.	No	Passive sentence; the agents are in a prepositional phrase (by ...), not a direct object
Google is headquartered in Mountain View, California.	No	The verb is a passive participle followed by a preposition; there is no direct object
Sundar Pichai is the CEO of Google.	No	The relation is a noun (CEO), not a verb
Google acquired YouTube in 2006.	Yes	Simple active sentence: subject, verb, direct object
Google launched the Knowledge Graph in 2012.	Partly	Object reduced to the head word Graph; the compound Knowledge is lost

The analysis shows three kinds of errors. First, passive voice hides the logical subject and object. Second, relations expressed by nouns (CEO of) or by verb plus preposition (headquartered in) are not covered by a verb-object rule. Third, taking only the head word of a phrase loses multi-word names. A better extractor would use noun-chunk or entity spans for the arguments and would add patterns for the passive and for noun-based relations.

4.7 Interactive Graph (PyVis)

The PyVis version of the graph is saved as an HTML file (kg.html) and displayed inside the notebook. Nodes can be dragged and the layout reacts to the movement, which makes it easy to

inspect dense areas. The screenshots of both graphs, the NER output and the triples are collected in the output screenshots PDF, and the demo video shows the interactive graph in use.

4.8 Complete Named Entity Output

Table 7: All entity mentions returned by spaCy for the input text

No.	Entity text	Label	Sentence
1	Google	ORG	1
2	Larry Page	PERSON	1
3	Sergey Brin	PERSON	1
4	1998	DATE	1
5	Google	ORG	2
6	Mountain View	GPE	2
7	California	GPE	2
8	Sundar Pichai	PERSON	3
9	Google	ORG	3
10	Google	ORG	4
11	YouTube	ORG	4
12	2006	DATE	4
13	Google	ORG	5
14	the Knowledge Graph	ORG	5
15	2012	DATE	5

The output includes the article in "the Knowledge Graph" because spaCy treats it as part of the entity span. A normalisation step that removes leading articles would give the node name Knowledge Graph used in the curated triples. The label ORG is also questionable for a product, which shows that a small pre-trained model does not always follow the type system of the target knowledge graph.

5. Case Study: Google Knowledge Graph

5.1 Overview

The Google Knowledge Graph was launched in 2012. It is a large database of entities and the relationships between them. It is used to show Knowledge Panels, direct answers and related entities in Google Search and in Google Assistant.

5.2 Role of information extraction

- Sources: Wikipedia, Wikidata, the CIA World Factbook, licensed data, structured markup on web pages, and text from web documents.
- NER and entity linking identify entities in web text and connect each mention to a single unique entity.
- Relation extraction at web scale, using machine learning, finds new facts from text.
- Knowledge fusion combines facts from many sources and uses confidence scores to resolve conflicts.
- Continuous updating keeps the graph current as new text is published.

5.3 Our pipeline compared with Google KG

Aspect	Our pipeline	Google Knowledge Graph
Scale	1 short text, 7 triples	Billions of facts
Data sources	Single plain text	Web, Wikipedia, Wikidata, structured data
Extraction method	spaCy NER + rule-based dependency rules	Machine learning and structured data
Entity linking	Not implemented	Core component
Fusion / validation	None	Multi-source fusion with confidence scores
Output	NetworkX / PyVis graph	Knowledge Panels in Search

5.4 History and Evolution

Google bought the company Metaweb in 2010 and with it the Freebase database, a large collaboratively built knowledge base. In May 2012 Google announced the Knowledge Graph with the slogan "things, not strings", and stated at that time that the graph contained more than 500 million objects and more than 3.5 billion facts about them. Since then the graph has grown many times larger. In 2014 Google researchers described the Knowledge Vault, a system that combines facts extracted from web text, tables, page structure and human-annotated pages with prior knowledge from existing knowledge bases, and gives each fact a probability of being true.

Freebase was closed in 2016 and its data was moved to Wikidata, while Google continues to offer access to its graph through the Knowledge Graph Search API.

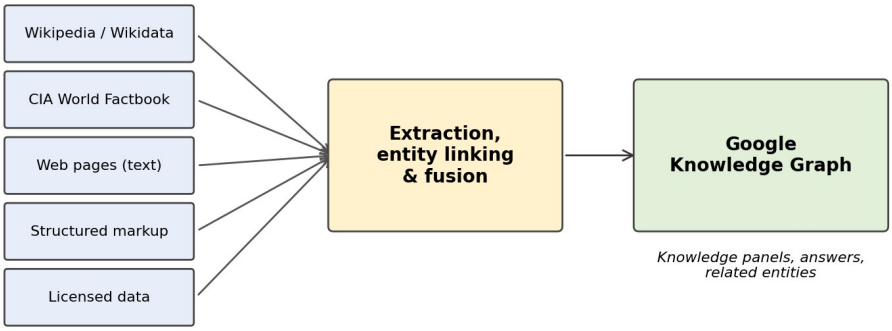


Figure 5: Sources feeding the Google Knowledge Graph (simplified)

5.5 How the Knowledge Graph is Used in Search

When a search query is recognised as a reference to an entity, Google shows a knowledge panel on the result page with a summary, key facts and links to related entities. For example, a search for a company shows its founders, headquarters and founding date. These values are not copied from one page: they are the facts stored for that entity in the graph. Each entity has a machine-generated identifier, so different names of the same entity lead to the same panel. The same facts are used by Google Assistant to answer spoken questions directly.

5.6 Role of Each IE Step in the Google Pipeline

Table 8: Mapping of information extraction steps to the Google Knowledge Graph

IE step	In this project	In Google Knowledge Graph
Entity recognition	spaCy en_core_web_sm	Large neural models over web pages and tables
Entity linking	Not done	Maps every mention to a unique entity identifier
Relation extraction	Verb-object rules	Learned extractors plus structured markup and wrappers
Fact validation	Manual check	Probabilistic fusion over many sources
Update	One run	Continuous crawling and re-extraction

5.7 Challenges Faced by Large Knowledge Graphs

- Accuracy: even a small error rate becomes millions of wrong facts at web scale.
- Freshness: facts such as the CEO of a company change, so old facts must be updated or retired.
- Ambiguity: many entities share a name, so linking errors merge or split entities.

- Coverage: knowledge about less popular people, places and languages is limited.
- Provenance: the system must remember where each fact came from in order to check it.

6. Limitations and Discussion

- Multi-word entities can be split, for example "Knowledge Graph" became "Graph" in automatic extraction.
- Rule-based extraction misses many facts, since it only handles simple subject-verb-object sentences. Relations such as "CEO of" and "headquartered in" were missed.
- No entity linking, so ambiguous names (Google the company vs. other uses) cannot be resolved.
- Small en_core_web_sm model has limited accuracy; for example, "the Knowledge Graph" was labelled ORG rather than a product.
- Scale: the toy pipeline works on one text, whereas real KGs need distributed, continuously updated pipelines.

These limitations show why large systems such as Google KG combine machine-learning-based extraction, entity linking, structured data and fusion, instead of relying on simple rules.

6.1 Future work

- Use a larger model (en_core_web_trf) or a transformer-based relation extraction model.
- Add entity linking against Wikidata.
- Extract triples from many Wikipedia pages and compare them with Wikidata facts.

6.2 Quality, Bias and Ethical Considerations

A knowledge graph presents facts with an air of authority, so errors and bias in the extracted data can mislead many users. Text sources over-represent some regions, languages and groups, so the resulting graph knows more about them. Facts about private persons raise privacy questions, and automatic extraction can reveal or combine personal information that was spread over many pages. For these reasons production systems record the source and confidence of every fact, allow corrections and removal requests, and combine automatic extraction with human review. In this project only public facts about a company were used.

6.3 Suggestions for Improving the Pipeline

- Use noun chunks or entity spans instead of single head words, so that Knowledge Graph stays whole.
- Add dependency patterns for passive sentences (was founded by) and for noun relations (CEO of).

- Merge entity mentions, for example by lower-casing and removing the article, before adding nodes.
- Link each entity to a Wikidata identifier to remove duplicates and add the type of the entity.
- Evaluate on a larger labelled data set instead of one hand-written text.

7. Conclusion

This project showed that information extraction is the foundation of knowledge graph construction: NER finds the entities, relation extraction finds the connections, and triples turn them into a graph. The implemented pipeline successfully converted a short text into a knowledge graph with 7 facts, and the comparison with Google Knowledge Graph showed that real systems need entity linking, machine learning at scale, multiple sources and knowledge fusion. The analysis of extraction errors also gives a clear view of why high-quality extraction is the hardest and most important step in building a knowledge graph.

8. Project Links

- GitHub repository: https://github.com/rakavirakesh-dev/kg-information-extraction
- Deliverables: Jupyter Notebook (.ipynb), report (PDF), output screenshots (PDF), demo video (MP4).

9. References

- Singhal, A. (2012). Introducing the Knowledge Graph: things, not strings. Google Official Blog.
- Hogan, A. et al. (2021). Knowledge Graphs. ACM Computing Surveys, 54(4).
- Jurafsky, D. and Martin, J. H. Speech and Language Processing (3rd ed. draft), chapters on Information Extraction.
- spaCy documentation: https://spacy.io
- NetworkX documentation: https://networkx.org
- PyVis documentation: https://pyvis.readthedocs.io
- Wikidata: https://www.wikidata.org
- Dong, X. et al. (2014). Knowledge Vault: A Web-Scale Approach to Probabilistic Knowledge Fusion. KDD 2014.
- Bollacker, K. et al. (2008). Freebase: A Collaboratively Created Graph Database for Structuring Human Knowledge. SIGMOD 2008.
- Vrandecic, D. and Kroetzsch, M. (2014). Wikidata: A Free Collaborative Knowledgebase. Communications of the ACM, 57(10).
- Auer, S. et al. (2007). DBpedia: A Nucleus for a Web of Open Data. ISWC 2007.

- Mintz, M. et al. (2009). Distant Supervision for Relation Extraction without Labeled Data. ACL 2009.
- Banko, M. et al. (2007). Open Information Extraction from the Web. IJCAI 2007.
- Mitchell, T. et al. (2018). Never-Ending Learning. Communications of the ACM, 61(5).
- Honnibal, M. and Montani, I. spaCy: Industrial-strength Natural Language Processing in Python. https://spacy.io

Appendix A: Complete Source Code

The complete notebook is available in the GitHub repository. The main cells are reproduced below.

Cell 1: Installation

```
!pip install spacy networkx pyvis
!python -m spacy download en_core_web_sm
```

Cell 2: Named Entity Recognition

```
import spacy
nlp = spacy.load("en_core_web_sm")

text = """
Google was founded by Larry Page and Sergey Brin in 1998.
Google is headquartered in Mountain View, California.
Sundar Pichai is the CEO of Google.
Google acquired YouTube in 2006.
Google launched the Knowledge Graph in 2012.
"""

doc = nlp(text)
for ent in doc.ents:
    print(ent.text, "->", ent.label_)
```

Cell 3: Triples and automatic extraction

```
triples = [
    ("Larry Page", "founded", "Google"),
    ("Sergey Brin", "founded", "Google"),
    ("Google", "headquartered in", "Mountain View"),
    ("Mountain View", "located in", "California"),
    ("Sundar Pichai", "CEO of", "Google"),
    ("Google", "acquired", "YouTube"),
    ("Google", "launched", "Knowledge Graph"),
]

auto_triples = []
for sent in doc.sents:
    for token in sent:
        if token.pos_ == "VERB":
            subj = [w.text for w in token.lefts if w.dep_ in ("nsubj",
"nsubjpass")]
            obj = [w.text for w in token.rights if w.dep_ in ("dobj", "attr")]
            if subj and obj:
                auto_triples.append((subj[0], token.lemma_, obj[0]))

print("Auto extracted:")
for t in auto_triples:
    print(t)
```

Cell 4: Static graph with NetworkX and Matplotlib

```
import networkx as nx
import matplotlib.pyplot as plt

G = nx.DiGraph()
for s, r, o in triples:
    G.add_edge(s, o, label=r)

plt.figure(figsize=(12, 8))
pos = nx.spring_layout(G, seed=42, k=1.5)
```

```
nx.draw(G, pos, with_labels=True, node_color="skyblue",
        node_size=3500, font_size=9, arrows=True)
nx.draw_networkx_edge_labels(G, pos,
        edge_labels=nx.get_edge_attributes(G, "label"), font_size=8)
plt.title("Knowledge Graph: Google")
plt.show()
```

Cell 5: Interactive graph with PyVis

```
from pyvis.network import Network

net = Network(notebook=True, directed=True, height="500px",
              width="100%", cdn_resources="in_line")
for s, r, o in triples:
    net.add_node(s, label=s)
    net.add_node(o, label=o)
    net.add_edge(s, o, label=r)

net.show("kg.html")
from IPython.display import HTML
HTML(open("kg.html").read())
```

Appendix B: Repository Structure and How to Run

Table 9: Files in the GitHub repository

File	Description
README.md	Project description, pipeline and instructions
kg_information_extraction.ipynb	Complete Jupyter Notebook with code and outputs
output_screenshots.pdf	Screenshots of the outputs
KG_Term_Project_Report.pdf	This report

Steps to run

- Open Google Colab (colab.research.google.com) and upload the notebook, or open it from GitHub.
- Run the first cell to install spaCy, NetworkX and PyVis and to download the model en_core_web_sm.
- Run the remaining cells from top to bottom: NER, triples, static graph, interactive graph.
- To use another text, replace the value of the variable text and edit the list of curated triples.

Appendix C: Glossary

Table 10: Terms used in this report

Term	Meaning
Entity	A real-world object such as a person, organisation, place or date
Triple	A fact written as (subject, relation, object)
NER	Named Entity Recognition: finding and labelling entities in text
Entity linking	Mapping a mention in text to a unique entry in a knowledge base
Relation extraction	Finding the relation between two entities in a sentence
Dependency parse	Tree showing the grammatical relation between words
Knowledge fusion	Merging facts from many sources into one consistent graph
Ontology	Schema that defines the types of entities and relations
Precision / Recall	Share of extracted facts that are correct / share of correct facts that were found